AUGUST 2026

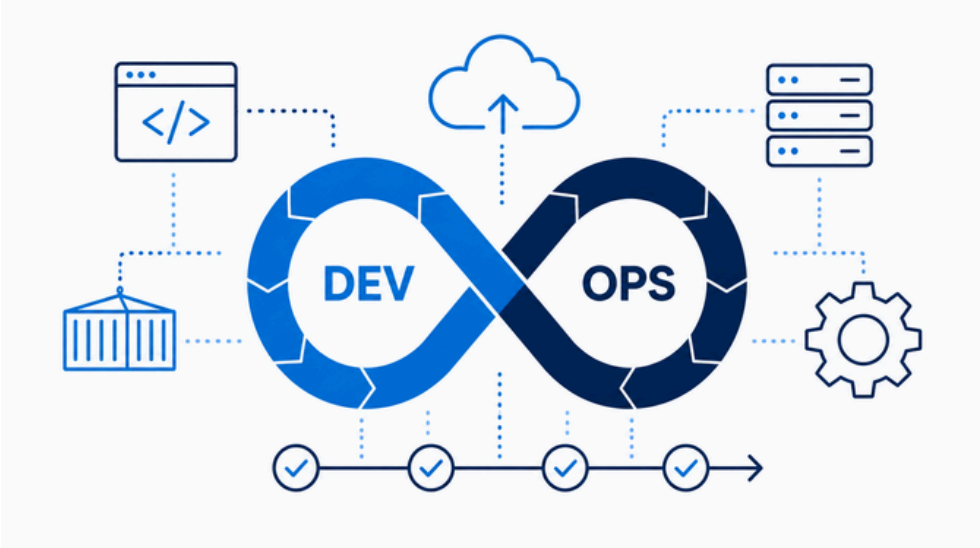
3-Tier DevSecOps

Clone-to-Running Deployment Guide

Reproduce the 3-Tier DevSecOps project from Git clone to a validated local orAWS EKS deployment

QUICK START + FULL CLOUD REPRODUCTION

Version 1.0



BY

Moaaz Essam

Important branch note
Asof20 August 2026,thecomplete DevSecOps implementation is on feature/ci-cd. The default main branch does not contain the full infrastructure, Helm, pipeline, and monitoring implementation.

Before you provision EKS
Setcluster_endpoint_public_access_cidrs to your current public IP (/32) and include the self-hosted agent public IP if it is different. If this is wrong, kubectl/Azure DevOps deployment access to the EKS API will fail.

1. Choose Your Goal

Path	Use when	What you need
A - LocalDocker Compose	You want to run and test the 3-tier app quickly	Git+ Docker Engine/Desktop + Docker Compose
B - Full AWS/EKS DevSecOps	You want to reproduce infrastructure, CI/CD, security, Kubernetes, monitoring and alerts	AWS + Terraform + Azure DevOps + self-hosted agent + kubectl + Helm + Slack webhook

2. Prerequisites

Requirement	Local	Full AWS/EKS	Notes
Git	Required	Required	Clonethe repository
Docker + Compose	Required	Required for local testing / CI build concepts	Docker Compose v2 preferred
AWS account	No	Required	Permissions to create VPC/EKS/ECR/IAM/S3/EC2/ELB resources
AWS CLI v2	No	Required	Authenticate Terraform/admin workstation and deployment agent
Terraform	No	Required	Project requires >= 1.15.0 and < 2.0.0
kubectl	No	Required	Use a version compatible with the EKS Kubernetes version; project baseline is 1.35
Helm 3	No	Required	Deploy AWS Load Balancer Controller, app chart, and monitoring
Azure DevOps	No	Required	Pipeline + OIDC AWS service connection
Rocky Linux self-hosted agent	No	Required	Pool name expected by YAML: eks-deploy-pool
Slack webhook	No	Required for Stage 8 Slack alerts	Store as a secret pipeline variable

3. Clone the Repository

```
git clonehttps://github.com/Moaazua/moaaz-3tier-devsecops.git
cd moaaz-3tier-devsecops
git checkout feature/ci-cd
git status
```

Confirm you can see helm-chart/, infrastructure/, monitoring/, pipelines/, compose.yaml, and azure-pipelines.yml before continuing.

4. Path A - Run Locally with Docker Compose

This path proves the application itself before any cloud provisioning.

1. Create a local environment file from .env.example.
2. Replace the example PostgreSQL password with a non-default local password.
3. Build and start the database, backend, and frontend.
4. Validate container health and the frontend/API endpoints.
5. Create and delete a note in the UI and confirm the API response changes.

```
cp .env.example .env
# edit .env and change POSTGRES_PASSWORD

docker compose up -d --build
docker compose ps

# Frontend
# http://localhost:8080

# Backend API
# http://localhost:5000/api/messages

# Stop local environment
docker compose down
```

Expected result
PostgreSQLbecomes healthy first, then the Flask backend, then the React/Nginx frontend. The UI and /api/messages endpoint should reflect the same data.

5. Path B - Prepare Account-Specific Values

The repository is an as-built portfolio project, so several values are intentionally specific to the original AWS/Azure DevOps environment. Replace them before running the full deployment.

Value to review	Where	What to change
AWS account / ECR registry	pipelines/stages/05-push-ecr.yml, helm-chart/values.yaml Terraform tfvars and pipeline stages 05-08	Replacethe original account ID and repository URLs with your account/ECR URLs
AWS region	Terraform tfvars	Keep eu-central-1 or update all references consistently
Project/environment		Changing names changes VPC/EKS/IAM/ECR resource names; then update hard-coded pipeline cluster/repo names
EKS API CIDR	infrastructure/terraform.tfvars	Use your admin IP and self-hosted agent public IP as /32 entries
Azure DevOps role	azure_devops_role_name	Set the name of the IAM role created for your Azure DevOps OIDC connection
Service connection name	pipeline YAML	YAML expects aws-ecr-oidc unless you rename references
Agent pool	stages 06-08	YAML expects eks-deploy-pool unless you rename references

6. Authenticate to AWS

```
aws sts get-caller-identity

# If you do not already have a working profile:
aws configure --profile moaaz-dev
export AWS_PROFILE=moaaz-dev
aws sts get-caller-identity
```

Troubleshooting rule
Donotdebug Terraform first when it says no valid credential sources. Prove the AWS CLI identity with aws sts get-caller-identity, then return to Terraform.

7. Bootstrap the Terraform Remote State

The bootstrap layer creates a versioned, encrypted, public-blocked S3 bucket. Its name is derived from project, environment, and your AWS account ID.

```
terraform -chdir=infrastructure/bootstrap/s3-backend init
terraform -chdir=infrastructure/bootstrap/s3-backend plan -out=s3-backend.tfplan
terraform -chdir=infrastructure/bootstrap/s3-backend apply s3-backend.tfplan

STATE_BUCKET=$(terraform -chdir=infrastructure/bootstrap/s3-backend output -raw state_bucket_name)
echo"$STATE_BUCKET"
```

Persistent foundation
Thestate buckethasprevent_destroy enabled. Keep the bootstrap state safe and do not treat this bucket as disposable runtime infrastructure.

8. Create the Persistent ECR Stack

```
cpinfrastructure/stacks/ecr/terraform.tfvars.example          infrastructure/stacks/ecr/terraform.tfvars

terraform -chdir=infrastructure/stacks/ecr init          -backend-config="bucket=$STATE_BUCKET"-backend-
config="key=dev/ecr.tfstate"       -backend-config="region=eu-central-1"

terraform -chdir=infrastructure/stacks/ecr plan -out=ecr.tfplan
terraform -chdir=infrastructure/stacks/ecr apply ecr.tfplan
```

Record the frontend and backend ECR repository URLs and replace the project-specific ECR URLs in Stage 5 and helm-chart/values.yaml.

9. Create Azure DevOps OIDC Access to AWS

The runtime Terraform expects the Azure DevOps IAM role to exist before the EKS cluster is applied. Create the role and service connection before Step 10.

6. In Azure DevOps, create or prepare the project that will run azure-pipelines.yml.
7. Create an AWS service connection using OIDC/federation and name it aws-ecr-oidc (or update the YAML).
8. Create the matching AWS IAM role using the issuer and subject generated for your Azure DevOps organization/project/service connection. Do not copy the original project issuer/subject.
9. Grant the role permission to authenticate to ECR and push images to the two project repositories.
10. Set azure_devops_role_name in infrastructure/terraform.tfvars to this role name. Terraform will later add eks:DescribeCluster permission plus an EKS access entry with cluster-admin access.

Security note
UsetheexactOIDC issuer/subject generated for your own Azure DevOps connection. Do not put long-lived AWS access keys in pipeline variables.

10. Configure Runtime Terraform Variables

```
cp infrastructure/terraform.tfvars.exampleinfrastructure/terraform.tfvars

# Edit infrastructure/terraform.tfvars before apply.
# Minimum items to review:
# - aws_region
# - project_name / environment
# - cluster_endpoint_public_access_cidrs
# - node sizing
# - azure_devops_role_name
```

CIDR checkpoint
Thisis thestepwhere you must update EKS public access CIDRs. Add the public IP of the machine running kubectl and the Rocky Linux deployment agent if it uses a different egress IP.

11. Initialize and Apply the Runtime Infrastructure

```
terraform -chdir=infrastructure init          -backend-config="bucket=$STATE_BUCKET"
-backend-config="key=dev/terraform.tfstate"       -backend-config="region=eu-central-1"

terraform -chdir=infrastructure fmt -check
terraform -chdir=infrastructure validate
terraform -chdir=infrastructure plan -out=runtime.tfplan
terraform -chdir=infrastructure apply runtime.tfplan
```

Expected runtime components include VPC networking, private EKS nodes, IAM/security resources, Pod Identity Agent, VPC CNI, CoreDNS, kube-proxy, AWS EBS CSI driver, and IAM/Pod Identity resources for the AWS Load Balancer Controller.

12. Configure kubectl and Validate EKS

```
CLUSTER_NAME="moaaz-3tier-devsecops-dev-eks"
AWS_REGION="eu-central-1"

aws eks update-kubeconfig --region "$AWS_REGION" --name "$CLUSTER_NAME"
kubectl cluster-info
kubectl get nodes -o wide
aws eks list-addons --cluster-name "$CLUSTER_NAME" --region "$AWS_REGION"
kubectl get csidriver ebs.csi.aws.com
```

Do not continue if this fails
FixEKSaccess,publicCIDRs,node readiness, VPC CNI, or EBS CSI issues before deploying the application.

13. Install and Validate AWS Load Balancer Controller

Terraform creates the controller IAM role/policy and Pod Identity association, but the controller workload is installed with Helm. Install it before the application pipeline so the Kubernetes Ingress can receive an ALB.

```
VPC_ID=$(aws eks describe-cluster      --name "$CLUSTER_NAME"--region "$AWS_REGION"   --query
'cluster.resourcesVpcConfig.vpcId'        --output text)

helm repo add eks https://aws.github.io/eks-charts
helm repo update

helm upgrade --install aws-load-balancer-controller        eks/aws-load-balancer-controller      -n kube-system    --set
clusterName="$CLUSTER_NAME"-set region="$AWS_REGION"   --set vpcId="$VPC_ID"      --set
serviceAccount.create=true      --set serviceAccount.name=aws-load-balancer-controller

kubectl rollout status deployment/aws-load-balancer-controller -n kube-system --timeout=5m
kubectl getpods -nkube-system -l app.kubernetes.io/name=aws-load-balancer-controller
```

Required validation
Thecontrollermustbehealthy before Stage 6. If it is not, inspect deployment events and Pod Identity/IAM first; do not troubleshoot the application ingress yet.

14. Prepare the Rocky Linux Self-Hosted Azure DevOps Agent

11. Create an Azure DevOps agent pool named eks-deploy-pool.
12. Provision/register a Rocky Linux self-hosted agent using the current Azure DevOps agent package shown by your pool UI.
13. Install AWS CLI v2, kubectl, and Helm on the agent host.
14. Verify those tools are in the PATH of the actual agent service user, not only your interactive shell.
15. From the agent host, verify internet access and that the EKS API CIDR allows its public egress IP.

```
aws --version
kubectl version --client
helm version

# After the role/service connection is used by the pipeline,
# Stages 6-8 will run on pool: eks-deploy-pool
```

15. Configure Azure DevOps Secret Variables

Variable	Secret?	Used by
POSTGRES_PASSWORD	Yes	Stage 6 Helm deployment, injected into postgres.auth.password
SLACK_WEBHOOK_URL	Yes	Stage 8; stored as Kubernetes Secret slack-webhook in monitoring namespace

Do not commit either value to Git. Mark both as secret variables in Azure DevOps.

16. Create and Run the Azure DevOps Pipeline

Create a pipeline from the repository azure-pipelines.yml. The current implementation executes eight stages in order:

```
1  Application Tests
2  Trivy Security Scan
3  Docker Build
4  Container Image Scan
5  Push Images to Amazon ECR
6  Deploy to Amazon EKS
7  Post-Deployment Validation
8  Deploy & Validate Monitoring
```

Stages 1-5 can run on Microsoft-hosted Ubuntu agents. Stages 6-8 explicitly use the self-hosted eks-deploy-pool. Stage 5 and the EKS/monitoring stages use the aws-ecr-oidc service connection.

17. Validate the Application

```
kubectl get nodes
kubectl get pods -n note-app-owide
kubectl get svc -n note-app
kubectl get pvc -n note-app
kubectl get ingress -n note-app
helm status note-app -n note-app

ALB_HOST=$(kubectl get ingress note-app-ingress -n note-app  -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')

echo "http://$ALB_HOST"
curl -I "http://$ALB_HOST/"
curl "http://$ALB_HOST/api/messages"
```

Expected result
Frontend returnsHTTP 200, the backend /api/messages endpoint returns HTTP 200, PostgreSQL is Running with a Bound PVC, and the ingress has an AWS ALB hostname.

18. Validate Monitoring

```
kubectl get pods  -nmonitoring
kubectl get prometheusrule note-app-alerts -n monitoring
kubectl get alertmanagerconfig slack-alerts -n monitoring
helm status monitoring -n monitoring

# Grafana local access
kubectl port-forward svc/monitoring-grafana -n monitoring 3000:80 #
Open http://localhost:3000

# Prometheus local access (run in another terminal when needed)
kubectl port-forward svc/monitoring-kube-prometheus-prometheus  -nmonitoring9090:9090
```

Validate that the custom note-app rules exist and that a controlled test alert produces both FIRING and RESOLVED messages in Slack. Restore the application immediately after the test.

19. Fast Troubleshooting Checklist

Symptom	First checks
Terraform says no validcredentials	aws sts get-caller-identity, verify AWS_PROFILE / environment credentials
kubectl cannot reach cluster	EKS public CIDRs, kubeconfig, aws sts identity, EKS access entry
Nodes not Ready / aws-node problems	kubectl get pods -n kube-system; describe events; verify VPC CNI
PVC Pending	Pod Identity/IAM EBS CSI add-on, StorageClass ebs-gp3, node/AZ scheduling
Ingress has no ALB	AWS Load Balancer Controller rollout, Pod Identity/IAM, ingress annotations, subnet tags
Stage 6 cannot find tools	Check aws/kubectl/helm in PATH for the Azure DevOps agent service user
Stage 5 ECR push fails	OIDC service connection, IAM role permissions, ECR URL/account/region
Slack alerts missing	SLACK_WEBHOOK_URL secret, slack-webhook Kubernetes Secret, AlertmanagerConfig selector/labels

20. Shutdown / Cost-Control Procedure

The normal lifecycle removes only the runtime infrastructure and keeps the ECR stack and S3 remote state.

```
# Optional: stop local environment
docker compose down

# Destroy AWS runtime only
terraform -chdir=infrastructure plan -destroy -out=destroy.tfplan
terraform -chdir=infrastructure apply destroy.tfplan

# Verify runtime state is empty
terraform -chdir=infrastructure state list
```

Do not delete the persistent layers in the normal workflow
Donotdestroy infrastructure/stacks/ecrorthe bootstrap S3 backend when you only want to stop AWS runtime cost. Keeping them preserves images and Terraform state for the next apply.

21. Publishing / Forking Checklist

- Replace the original AWS account ID and ECR URLs.
- Use your own OIDC issuer/subject and IAM role.
- Use your own POSTGRES_PASSWORD and Slack webhook.
- Review EKS public API CIDRs before every new environment or network change.
- Confirm service connection and agent pool names match azure-pipelines.yml.
- Never commit .env, terraform.tfvars, AWS credentials, kubeconfig, or webhook URLs.
- If the complete implementation is later merged to main, clone main instead of feature/ci-cd and update public documentation links accordingly.

22. References

Repository: https://github.com/Moaazua/moaaz-3tier-devsecops

Current complete branch: https://github.com/Moaazua/moaaz-3tier-devsecops/tree/feature/ci-cd

Guide basis:currentrepositorysourceplustheactualbuild,validation,troubleshooting,monitoring,andteardown workflow used during the project.